biggest fragments and try gluing them into place, and eventually piece together the vase. At this points it's important to realize the limitations of our analogy. In a real LT coding scheme, the bag of fragments would be bottomless, so that we could always reach in and grab a new fistful of fragments, each different from the previous ones, and each containing new pieces of the vase that would be useful in reconstructing it. At this point, the reader might be thinking that this process sounds pretty wasteful, since it would end up getting lots of duplicate fragments. Indeed, it's hard to see how this wouldn't lead to a lot of overhead in the protocol. Obviously there needs to be some degree of redundancy/overhead in any robust system like this, but we should distinguish between redundant data and wasteful overhead. By wasteful overhead, we mean that some of the bits we are sending are not the actual data itself, but rather the protocol that describes the fragments and how to assemble them. While such information is needed to make the system work, we want to keep it to an absolute minimum. After all, if we had to download 30 megabytes worth of 3 megabyte chunks in order to reliably assemble a 5 megabyte file, that would be a pretty bad outcome, since the system would only be 5/30 = 16.6% efficient. This leads to the second surprising property of LT codes: in practice, they can achieve efficiency rates of upwards of 85%, as shown in the following figure, which shows that in order to download 2,048 "symbols" worth of data with a high probability of success, we generally will only need to download 2,300 symbols worth of LT chunks. As we continue to download more chunks, the probability of successfully decoding the entire file approaches 100%. This means that the vast majority of our bandwidth is being put towards useful work: transferring the bits of data we want, not useless overhead that uses up our bandwidth without getting us closer to what we want– recon- structing the file. And yet, we still get these almost miraculous benefits: we can get file chunks from multiple sources, in any order, and every chunk is just as good/useful as any other chunk. No more stuck downloads, no more "rare" file chunks that we hope someone will make available– just data droplets, like water from a fountain, that we can slurp up from anywhere until our glass is full and we have the file! But how will we know that we have the right file? That part is easy: we store the SHA3-256 hash of the original image file in the art registration ticket on the blockchain, and we keep getting new LT chunks until the reconstructed file hashes to this exact same value. LT Codes were originally designed for a broadcaster-subscriber model, in which the broadcaster (for example, the NFL streaming a live game, or Steam serving a video game demo to users) has a full copy of the file/stream, and generates new LT chunks on the fly in an "endless stream" of chunks, and 99